Arbeidskrav – Python-programmering

Grunnleggende programmering og objektorientert programmering

20 oppgaver

Individuelt arbeid

Formål og læringsmål

Målet med arbeidskravet er å vise at dere kan utvikle egne Python-programmer, bruke riktig programmeringslogikk og kombinere flere programmeringsteknikker for å løse praktiske problemer.

Etter gjennomført arbeidskrav skal dere kunne:

- Bruke variabler, datatyper, operatorer, input() og print().
- Arbeide med tekststrenger, lister og typekonvertering.
- Bruke if, elif og else til å styre programflyt.
- Bruke for- og while-løkker.
- Lage funksjoner med parametere og returverdier.
- Opprette klasser, objekter, konstruktører og metoder.
- Kombinere klasser, funksjoner, lister og løkker i større programmer.
- Teste, feilsøke og dokumentere egen kode.

Oppgavedel

Del 1 – Grunnleggende programmering

Oppgave 1 – Personlig informasjon

Lag et program som ber brukeren skrive inn navn, alder og bosted.

Programmet skal skrive ut informasjonen i en ryddig setning og beregne hvor gammel personen blir om fem år.

Krav: Bruk variabler, input(), print() og typekonvertering.

Oppgave 2 – Enkel kalkulator

Lag et program som ber brukeren skrive inn to tall.

Programmet skal beregne og skrive ut summen, differansen, produktet og divisjonen av tallene.

Krav: Bruk float() og de fire regneoperatorene. Programmet skal håndtere divisjon på null.

Oppgave 3 – Temperaturkalkulator

Lag et program som konverterer temperatur fra Celsius til Fahrenheit og Kelvin.

Bruk formlene:

F = C * 9/5 + 32

K = C + 273.15

Programmet skal skrive ut begge resultatene med to desimaler.

Oppgave 4 – Tekstbehandling

Lag et program som ber brukeren skrive inn fornavn og etternavn.

Programmet skal skrive ut fullt navn, navnet med store bokstaver, antall bokstaver uten mellomrom og de tre første bokstavene i fornavnet (eller hele fornavnet dersom det er kortere).

Krav: Bruk strengoperasjoner som .upper(), len() og slicing.

Oppgave 5 – Handleliste

Lag et program som oppretter en liste med fem matvarer.

Programmet skal skrive ut alle varene, legge til én ny vare, fjerne én eksisterende vare og vise antall varer som er igjen.

Krav: Bruk en liste, .append(), .remove() og en for-løkke.

Del 2 – Betingelser og løkker

Oppgave 6 – Karakterkalkulator

Lag et program som ber brukeren skrive inn en poengsum mellom 0 og 100.

Programmet skal gi en karakter basert på følgende tabell:

Poeng	Karakter
90–100	A
80–89	B
70–79	C
60–69	D
50–59	E
0–49	F

Krav: Bruk if/elif/else. Ugyldige poengsummer skal avvises.

Oppgave 7 – Partall og oddetall

Lag et program som ber brukeren skrive inn et positivt heltall.

Programmet skal bruke en for-løkke til å skrive ut alle partall fra 1 til det oppgitte tallet, telle hvor mange partall som finnes og beregne summen av dem.

Krav: Bruk %-operatoren og en for-løkke.

Oppgave 8 – Gjettelek

Lag et program hvor brukeren skal gjette et hemmelig tall mellom 1 og 100. Det hemmelige tallet kan være 37.

Programmet skal fortsette til brukeren gjetter riktig, gi beskjed om tallet er for høyt eller for lavt, og telle antall forsøk.

Krav: Bruk while, if/elif/else og en teller.

Oppgave 9 – Analyse av talliste

Lag et program som inneholder en liste med minst ti heltall, inkludert positive tall, negative tall og null.

Programmet skal bruke en for-løkke til å telle positive, negative og nullverdier. Det skal også beregne summen og gjennomsnittet av tallene.

Krav: Bruk løkker og betingelser. Ikke bruk sum() for å beregne summen.

Oppgave 10 – Enkel minibank

Lag et program med en startsaldo på 1000 kr og følgende meny:

--- MINIBANK ---

1. Vis saldo

2. Sett inn penger

3. Ta ut penger

4. Avslutt

Programmet skal kjøre til brukeren velger å avslutte.

Krav: Bruk while, if/elif/else og kontroller at uttak ikke overstiger saldoen eller at beløp er negative.

Del 3 – Funksjoner og parametere

Oppgave 11 – Kalkulator med funksjoner

Videreutvikle kalkulatoren fra oppgave 2.

Lag fire funksjoner: addisjon(a, b), subtraksjon(a, b), multiplikasjon(a, b) og divisjon(a, b).

Funksjonene skal returnere resultatene. Lag en meny hvor brukeren velger hvilken beregning som skal utføres.

Krav: Bruk parametere, return, while og if/elif/else. Divisjon på null skal håndteres.

Oppgave 12 – Finne største og minste tall

Lag en funksjon finn_storste(tall_liste) og en funksjon finn_minste(tall_liste).

Funksjonene skal ta imot en liste med tall og returnere henholdsvis det største og det minste tallet.

Krav: Bruk for-løkker og if-tester. Ikke bruk max() eller min(). Du kan forutsette at listen ikke er tom.

Oppgave 13 – Passordkontroll

Lag en funksjon sjekk_passord(passord) som returnerer True dersom passordet oppfyller følgende krav:

- Minst åtte tegn.
- Inneholder minst ett tall.
- Inneholder minst én stor bokstav.

Ellers skal funksjonen returnere False.

Brukeren skal kunne prøve på nytt til et gyldig passord er skrevet inn.

Krav: Bruk funksjon, parameter, return, for, while og if. Du kan bruke .isdigit() og .isupper().

Oppgave 14 – Studentresultater

Lag et program som lar brukeren registrere navn og poengsum for fem studenter.

Lagre navnene i én liste og poengsummene i en annen liste.

Lag funksjonene beregn_snitt(poeng) og finn_beste(navn, poeng). Den første skal returnere gjennomsnittet, mens den andre skal returnere navnet på studenten med høyest poengsum.

Krav: Bruk funksjoner med parametere, lister, for-løkker og return.

Del 4 – Klasser og objekter

Oppgave 15 – Personregister med klasser

Lag en klasse Person med attributtene navn og alder.

Klassen skal ha en konstruktør __init__() og en metode vis_info() som skriver ut personens informasjon.

Opprett minst fem personobjekter og lagre dem i en liste. Bruk en for-løkke til å vise alle personene. Programmet skal også vise om hver person er myndig.

Krav: Bruk klasse, objekter, self, konstruktør, metode, liste, for og if.

Oppgave 16 – Bankkonto som klasse

Lag en klasse Bankkonto med attributtene eier og saldo.

Klassen skal ha metodene sett_inn(belop), ta_ut(belop) og vis_saldo().

Opprett et bankkontoobjekt og lag en meny hvor brukeren kan sette inn penger, ta ut penger, vise saldo og avslutte programmet.

Krav: Bruk klasser, metoder med parametere, while, if/elif/else og kontroll av ugyldige beløp.

Oppgave 17 – Handlekurv med objekter

Lag en klasse Produkt med attributtene navn, pris og antall.

Klassen skal ha en metode vis_info() og en metode beregn_verdi() som returnerer pris * antall.

Opprett minst fire produktobjekter i en liste.

Lag en funksjon beregn_total(produkter) som går gjennom listen og returnerer samlet verdi av produktene.

Programmet skal vise hvert produkt, antall, pris og totalbeløpet.

Krav: Bruk klasser, objekter, metoder, funksjon med parameter, return, liste og for-løkke.

Del 5 – Kombinasjonsoppgaver

I denne delen skal dere kombinere flere av temaene fra arbeidskravet i mer komplette programmer.

Oppgave 18 – Biblioteksystem

Lag et program som håndterer bøker på et lite bibliotek.

Opprett en klasse Bok med attributtene tittel, forfatter og tilgjengelig. Nye bøker skal automatisk være tilgjengelige.

Klassen skal ha metodene laan_ut(), lever_tilbake() og vis_info().

Programmet skal inneholde minst fem bokobjekter og følgende meny:

--- BIBLIOTEK ---

1. Vis alle bøker

2. Lån en bok

3. Lever tilbake en bok

4. Vis tilgjengelige bøker

5. Avslutt

Krav: Bruk while til menyen, for til å søke gjennom listen og if til å kontrollere tilgjengelighet. En bok skal ikke kunne lånes ut dersom den allerede er utlånt.

Oppgave 19 – Lagerstyringssystem

Lag et program som håndterer lagerbeholdningen i en butikk.

Opprett en klasse Produkt med navn, pris og antall. Lag metodene selg(antall), fyll_pa(antall) og vis_info().

Programmet skal ha en liste med minst fire produkter og følgende meny:

--- LAGERSTYRING ---

1. Vis produkter

2. Selg produkt

3. Fyll på lager

4. Beregn lagerverdi

5. Avslutt

Lag også funksjonen beregn_lagerverdi(produkter).

Krav: Bruk klasser, metoder, funksjoner, lister, while, for og if. Programmet skal forhindre salg av flere varer enn det finnes på lageret.

Oppgave 20 – Studentadministrasjon

Lag et enkelt studentadministrasjonssystem som kombinerer temaene fra hele arbeidskravet.

Opprett en klasse Student med attributtene navn, alder og karakter. Klassen skal ha en konstruktør og en metode vis_info().

Programmet skal lagre studentobjekter i en liste og ha følgende meny:

--- STUDENTADMINISTRASJON ---

1. Registrer student

2. Vis alle studenter

3. Søk etter student

4. Fjern student

5. Beregn gjennomsnittskarakter

6. Finn studenten med høyest karakter

7. Avslutt

Krav: Bruk en while-løkke for menyen, for-løkker til å behandle studentene, if/elif/else til valg og kontroll, og egne funksjoner for gjennomsnitt og høyeste karakter. Karakteren må være mellom 1 og 6. Programmet skal håndtere en tom studentliste.

Innleveringskrav

Dere skal levere én samlet ZIP-fil med følgende innhold:

- En Python-fil for hver oppgave, navngitt oppgave01.py til oppgave20.py, eller en Python-fil med alle oppgavene fordelt i seksjoner.